Cost-Aware Entity Resolution: When Is a Large Language Model Worth Paying For?

D.1 Project Pre-Proposal, COMP 8157 Advanced Database Topics (Fall 2026)

Shashang Patel
School of Computer Science
University of Windsor
Windsor, Canada
patel3cd@uwindsor.ca

Nauman Syed
School of Computer Science
University of Windsor
Windsor, Canada
syed9i@uwindsor.ca

***Abstract*—Large language models (LLMs) resolve entities with high accuracy and little training data, but every candidate pair costs a model call, while classical matchers run almost for free. We propose a cross-paradigm, cost-accounted evaluation of end-to-end entity resolution pipelines, from rule-based and probabilistic matchers to fine-tuned language models and LLMs, together with a confidence-based cascade that sends only uncertain pairs to an LLM. We will measure where each approach lies on a cost–accuracy frontier across public benchmarks, how much of the LLM's accuracy the cascade keeps, and how blocking and inconsistent pairwise decisions limit final quality.**

***Index Terms*—Entity resolution, entity matching, large language models, blocking, model cascades, cost-aware data integration**

I. Core Idea

Entity resolution (ER) decides which records refer to the same real-world entity, such as one product listed differently by two retailers. It is a core step when integrating e-commerce catalogs, customer data, and health records. Generative LLMs now match records with high accuracy and little labeled data [1], [2], but each candidate pair costs a model call, and even after blocking a realistic workload holds tens of thousands to millions of pairs. Established methods such as probabilistic linkage and feature-based classifiers, by contrast, run at almost no cost. Practitioners therefore lack evidence on a simple question: which pairs actually need an LLM, and how much accuracy is lost by not using one? We propose a systematic, cost-accounted comparison of complete ER pipelines, in which blocking methods are paired with matchers of increasing cost, together with a confidence-based cascade that resolves easy pairs with cheap matchers and escalates only the uncertain ones. We will measure accuracy, monetary and compute cost, and the consistency of the final entity clusters. The project combines Category 4 (a comparative study) with Category 2 (a new method).

II. Novelty Claim

Recent cost-aware work on LLM-based matching quietly assumes that the cheapest alternative to a strong LLM is a weaker LLM. Existing methods batch pairs into one prompt, route between small and large models, or learn a policy over LLM operations, but the candidates are always LLMs. We challenge this assumption directly. Fifty years of ER research produced matchers that need no model calls at all, beginning with the probabilistic linkage of Fellegi and Sunter [3], and to our knowledge no study has placed rule-based, probabilistic, supervised, fine-tuned language model, and LLM matchers on one cost axis that also charges supervised methods for the labels they need. Our claim is that for many datasets the cost–accuracy frontier is dominated by non-LLM methods, and that a cascade starting from these matchers keeps most of an LLM's accuracy while sending only a small fraction of pairs to it. A second assumption we question is that matching can be judged pair by pair on fixed candidate sets. Blocking bounds the recall any matcher can reach, and independent pairwise decisions can violate transitivity, forming clusters that no real entity supports. We therefore evaluate whole pipelines and measure whether inexpensive clustering can repair such violations without extra LLM calls. The new insight we expect to contribute is a measured trade-off: when an LLM is worth its price for ER, and how much of its benefit a cheap cascade retains.

III. Related Work and Positioning

The closest works study the cost of LLM-based matching. BatchER [4] batches several pairs into one prompt and selects demonstrations to cover them, and reports that batching is far cheaper than prompting pair by pair. ComEM [5] compares matching, comparing, and selecting strategies, addresses global consistency among records, and composes a cheaper LLM with a stronger one. CaRL-EM [6] learns a reinforcement-learning controller that chooses among LLM operators and model sizes to optimize a quality–cost objective, and Huang et al. [7] benchmark accuracy and cost across 46 LLM configurations. Our study is not a reproduction of these because the search space is different: all four operate within the LLM family, whereas we include matchers with no model calls, charge labeling cost, and vary blocking. ComEM obtains

consistency through additional LLM calls; we test whether post-hoc clustering achieves it for free.

Narayan et al. [1] first showed that foundation models perform data wrangling tasks, including matching, in few-shot settings. Peeters et al. [2] compared hosted and open LLMs across prompt designs and found that prompts must be tuned for each model and dataset, which motivates our use of a fixed prompt across all settings. The WDC Products benchmark [8] tests generalization to unseen entities.

On the classical side, Splink [9] implements Fellegi–Sunter linkage at scale, Magellan [10] supports feature-based matching end to end, DeepMatcher [11] explored deep architectures, and Ditto [12] fine-tunes pre-trained transformers at the price of GPU training and labeled data. Köpcke et al. [13] showed that product matching is far harder than bibliographic matching and released benchmarks we reuse. Papadakis et al. [14] survey blocking, and DeepBlocker [15] studies embedding-based blocking. Most matching studies, including DeepMatcher's benchmark suite [11], evaluate on pre-generated candidate pairs, so the effect of blocking on end-to-end cost is left out. Finally, FrugalGPT [16] showed that cascading from cheaper to stronger LLMs can approach the best model's quality at a fraction of the cost. We extend the cascade across paradigms, starting from classical matchers. None of these works measures where non-LLM matchers sit on the same cost frontier as LLMs, or how blocking and cluster consistency shift it.

IV. Expected Outcome and Feasibility

We will implement the pipelines in Python. Blocking uses token blocking, sorted neighborhood, and embedding-based k-nearest-neighbor search over a vector index (MongoDB $vectorSearch). Matchers are string-similarity rules, an unsupervised Fellegi–Sunter model in Splink, a Magellan-style random forest, Ditto, and two open LLMs of at most eight billion parameters served locally through Ollama with greedy decoding, plus one small hosted model with cached responses to obtain real monetary costs. Experiments use DBLP–ACM, DBLP–Scholar, Abt–Buy, Amazon–Google, and WDC Products, spanning easy to hard. In the cascade, each pair is scored first by the cheapest calibrated matcher; confident pairs are resolved immediately and the uncertain band is escalated, and sweeping the thresholds traces a cost–quality curve. We report pairwise and cluster-level F1, API cost, compute time, estimated labeling cost, number of LLM calls, blocking pair completeness and reduction ratio, and the rate of transitivity violations. The expected outcome is a cost–accuracy Pareto frontier per dataset, a categorized failure analysis showing which record types break each matcher, and a clear picture of when an LLM is worth paying for. A small Streamlit interface will let users pick a pipeline, move the escalation threshold, and watch F1 and a live cost counter update, which also serves as our demo.

References

[1] A. Narayan, I. Chami, L. Orr, S. Arora, and C. Ré, "Can foundation models wrangle your data?" *Proc. VLDB Endow.*, vol. 16, no. 4, pp. 738–746, 2022.

[2] R. Peeters, A. Steiner, and C. Bizer, "Entity matching using large language models," in *Proc. 28th Int. Conf. Extending Database Technology (EDBT)*, 2025, pp. 529–541.

[3] I. P. Fellegi and A. B. Sunter, "A theory for record linkage," *J. Amer. Statist. Assoc.*, vol. 64, no. 328, pp. 1183–1210, 1969.

[4] M. Fan, X. Han, J. Fan, C. Chai, N. Tang, G. Li, and X. Du, "Cost-effective in-context learning for entity resolution: A design space exploration," in *Proc. 40th IEEE Int. Conf. Data Engineering (ICDE)*, 2024, pp. 3696–3709.

[5] T. Wang et al., "Match, compare, or select? An investigation of large language models for entity matching," in *Proc. 31st Int. Conf. Computational Linguistics (COLING)*, 2025, pp. 96–109.

[6] C. Guo, M. C. A. Klein, and Z. Huang, "CaRL-EM: Cost-aware reinforcement learning for entity matching with LLMs," in *Proc. 64th Annu. Meeting Assoc. Computational Linguistics (ACL)*, 2026, pp. 27281–27293.

[7] D. Huang, P. Pai, and Z. Wang, "Entity matching with LLMs at scale: Accuracy, cost, and reasoning effects," in *Proc. Pacific-Asia Conf. Knowledge Discovery and Data Mining (PAKDD)*, LNCS vol. 16603, Springer, 2026.

[8] R. Peeters, R. C. Der, and C. Bizer, "WDC Products: A multi-dimensional entity matching benchmark," in *Proc. 27th Int. Conf. Extending Database Technology (EDBT)*, 2024, pp. 22–33.

[9] R. Linacre et al., "Splink: Free software for probabilistic record linkage at scale," *Int. J. Population Data Science*, vol. 7, no. 3, 2022.

[10] P. Konda et al., "Magellan: Toward building entity matching management systems," *Proc. VLDB Endow.*, vol. 9, no. 12, pp. 1197–1208, 2016.

[11] S. Mudgal et al., "Deep learning for entity matching: A design space exploration," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, 2018, pp. 19–34.

[12] Y. Li, J. Li, Y. Suhara, A. Doan, and W.-C. Tan, "Deep entity matching with pre-trained language models," *Proc. VLDB Endow.*, vol. 14, no. 1, pp. 50–60, 2020.

[13] H. Köpcke, A. Thor, and E. Rahm, "Evaluation of entity resolution approaches on real-world match problems," *Proc. VLDB Endow.*, vol. 3, no. 1, pp. 484–493, 2010.

[14] G. Papadakis, D. Skoutas, E. Thanos, and T. Palpanas, "Blocking and filtering techniques for entity resolution: A survey," *ACM Comput. Surv.*, vol. 53, no. 2, 2020.

[15] S. Thirumuruganathan et al., "Deep learning for blocking in entity matching: A design space exploration," *Proc. VLDB Endow.*, vol. 14, no. 11, pp. 2459–2472, 2021.

[16] L. Chen, M. Zaharia, and J. Zou, "FrugalGPT: How to use large language models while reducing cost and improving performance," arXiv:2305.05176, 2023.